

# AIAP Cohort - Workbook

## Claude Mastery in Action: Build & Deploy a Real Product (Pennywise) with Claude Design + Claude Code

### What You Will Do

**Time:** ~90–120 minutes (much of it is hands-off waiting while Claude designs and builds for you).

**The 4-step flow:** (1) Design Pennywise in Claude Design → (2) Send it to Claude Code and build the real web app → (3) Make it browser-first and add an onboarding flow → (4) Deploy it live on Netlify.

**What you'll walk away with:** A live, shareable URL for your own financial-tracker web app — designed, built, and deployed end-to-end without writing a single line of code.

**Why it matters:** This is the complete “idea → design → build → ship” loop that professional teams use. Once you have done it once, you can build and launch almost any web app the same way.

### What You Will Achieve by the End

- You will have written a single, detailed design prompt and used Claude Design to generate a high-fidelity, interactive prototype of Pennywise — a personal net-worth tracker.
- You will have sent that design to Claude Code (inside the Claude desktop app) and had it build a real, working web app from your prototype.
- You will have refactored the app to be browser-first — every user's data stays in their own browser, with no server and no database — and added a first-time onboarding flow.
- You will have deployed the finished app to a live URL on Netlify by dragging and dropping a single folder.

All of this is possible because the Claude ecosystem hands each tool the right context: Claude Design plans the look and feel, then Claude Code turns that plan into shippable software — and you stay in the driver's seat, just describing what you want.

### Before You Start: What You Need

Please complete all of the following before starting the activity.

**Heads-up — this is a paid-plan workbook**

Unlike the other workbooks, this workflow cannot be done on Claude's free tier. Claude Design and Claude Code on the best models are paid features, and they consume usage credits as they run.

One Claude Design run costs roughly \$5–\$10; the full Claude Code build after the design costs roughly \$12 (about \$20 including the design). Expect to spend in the region of \$20–\$30 to complete this end-to-end.

**Recommended setup:** a Claude Pro plan (from \$20/month) with extra usage credits topped up, so you don't get locked out mid-build when your weekly limit is reached. Treat the spend as the cost of shipping a real product, not a subscription.

### 1. A paid Claude plan (with usage credits)

- A Claude Pro (or higher) plan, signed in and ready. Claude Design and Claude Code are not available on the free plan.
- Top up usage credits in advance. In Claude, go to [Settings → Usage](#) to see your weekly limit and remaining credits. Once the weekly limit is hit, builds continue on usage credits — so add a small buffer before you start.

### 2. The Claude desktop app

- Download and install the [Claude desktop app](#) (Mac or Windows) and log in. **Claude Code runs through the desktop app's Code mode** — this is the non-terminal, click-to-use version shown in the session.

### 3. A Netlify account (free)

- Create a free account at [netlify.com](#). Netlify is the tool we use to publish your app to a live URL — it costs nothing for projects this size.

### 4. A modern browser and ~2 hours

- Use Chrome or any modern browser. Set aside about two hours — a lot of that time is simply letting Claude design and build while you do other things.

## Part 1: Design Pennywise in Claude Design

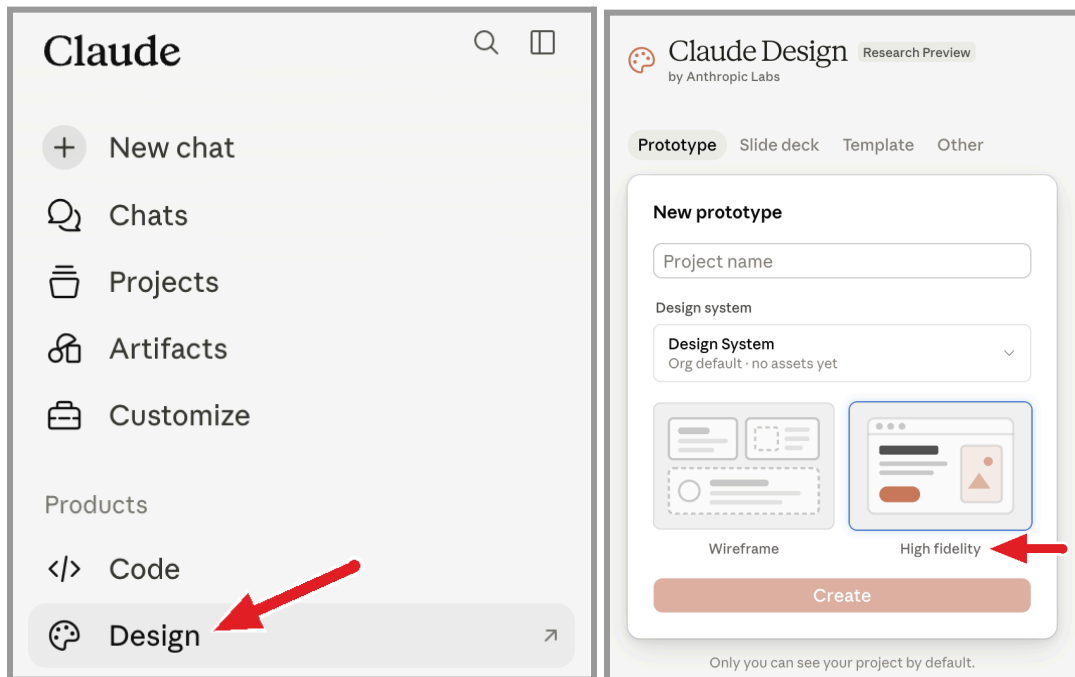
### 1.1 Open Claude Design and start a Prototype

- In [Claude](#), open **Claude Design**. You will see four options: Prototype, Slides, Design System, and others.
- Choose **Prototype**. This is the option that builds a real, clickable design you can play around in.

### 1.2 Name your project and choose High-Fidelity

- Give the project a name: **Pennywise**.

- When asked between **Wireframe** and **High-Fidelity**, choose **High-Fidelity**. A wireframe is just a rough skeleton; high-fidelity gives you a real, interactive design with colour, data, and working interactions.
- Do *not* select a design system for now. Leave it blank and click **Create**. You will land on a blank canvas — that is expected.



### 1.3 Write the design prompt

On the blank canvas, you only need to describe what you want to build. The more detailed your prompt, the better the questions Claude asks and the better the result. Paste the prompt below into the chat box.

**Prompt:** *I want you to build a design for a financial manager called Pennywise. This financial manager is targeted at high-income professionals and people who want to track their money on a weekly to quarterly basis. Their goal is to build their net worth — the clear focus is wealth increment, rather than just stock trading or anything else.*

*We are going for a very polished UI that is minimal, but at the same time provides enough value and enough detail to go into depth. The tool should be able to understand and track stocks, mutual funds, ETFs, gold, silver, and real estate, as well as other alternative investments like crypto or anything else the user might want to add.*

*The user should also have a selection of beautiful icons they can choose from, so they can add their own elements to track their net worth. The dashboard should be extremely high-fidelity, built around the idea that someone can get a glance of their net worth in under 10 seconds and immediately understand what has moved and what hasn't. It should not feel overwhelming — the UI should always feel close to the person, rather than a heavy,*

*data-driven financial dashboard. The idea is to make wealth-building a fun exercise rather than a cumbersome one.*

*Start by building the dashboard first, then the additional pages required to track the data. This should work as a real, interactive design — so if I add a mutual fund, the dashboard net worth should automatically update. Also include a drop-down for multi-currency and multiple languages that I can extend later; for now, use US Dollar and INR, and English.*

*My colours of choice are green and burnt orange — please design around those.*

Click send. Notice how long and specific the prompt is — that detail is what makes the output great.

### 1.4 Answer Claude's clarifying questions

On its first run, Claude Design asks a series of clarifying questions before it builds. The exact questions vary, but they will look like the ones below. Here is how Akshat answered each — feel free to adapt to your own taste.

| Question Claude asks                             | How Akshat answered (adapt freely)                                         |
|--------------------------------------------------|----------------------------------------------------------------------------|
| Do you want to upload a reference design?        | No — nothing yet, design from scratch.                                     |
| Overall aesthetic — warm, clean, or premium?     | Clean.                                                                     |
| What should the dashboard hero show first?       | Net worth, plus a clean trend chart.                                       |
| Default time view for tracking?                  | Let the user toggle; default to weekly.                                    |
| Which asset classes should feel most important?  | Stocks, crypto, gold and silver, and mutual funds — include all of them.   |
| Which pages should I build beyond the dashboard? | Asset-class detail, net-worth history, allocation breakdown, and settings. |
| How interactive should the prototype be?         | Fully interactive.                                                         |
| Which design direction do you prefer?            | Pick the one you like best (Akshat chose the cleanest option).             |
| Where should I explore multiple variations?      | The dashboard — that's the screen people look at most.                     |
| Which sample-data persona should I populate?     | An AI professional with a global portfolio; values in USD.                 |

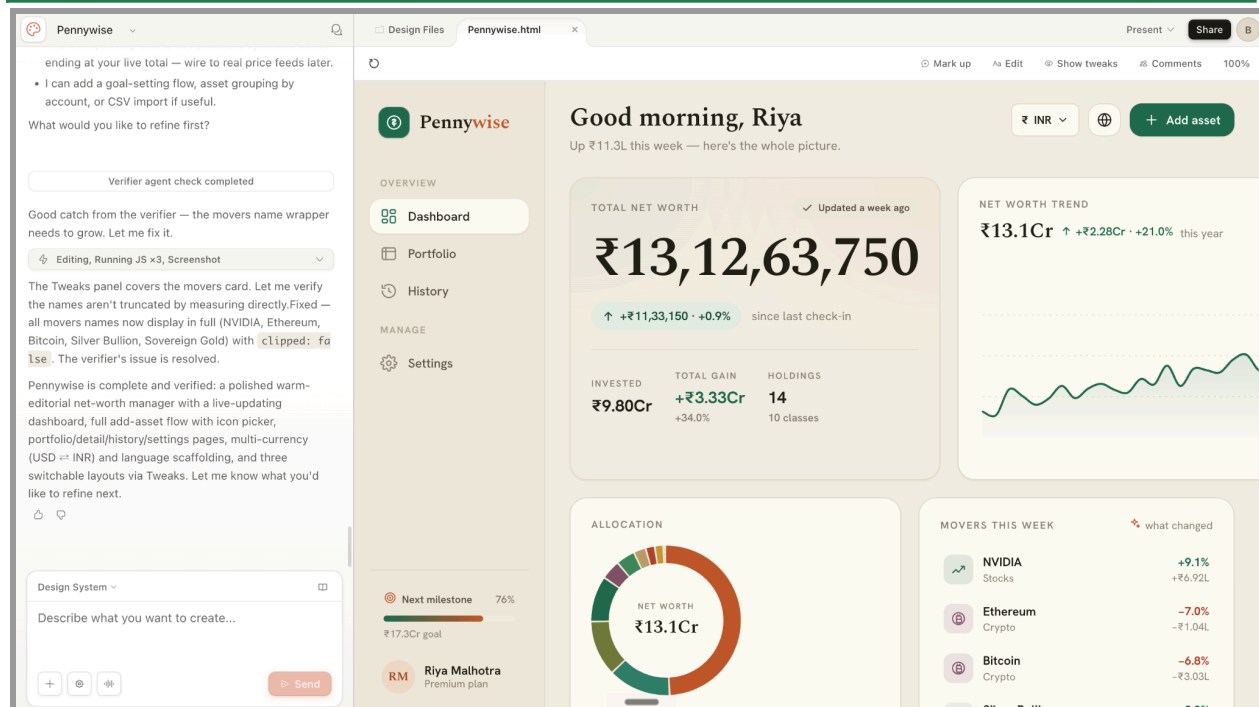
Once you have answered, click Continue.

### 1.5 Let it build

**Be patient — this is the slow, expensive step**

The first generation takes roughly 7–10 minutes and may cost \$5–\$10. Claude Design automatically uses the best model (an Opus model) for design quality — do not change it; lighter models are noticeably worse at design.

**Let it cook.** You do not need to watch it. Come back in 10 minutes.

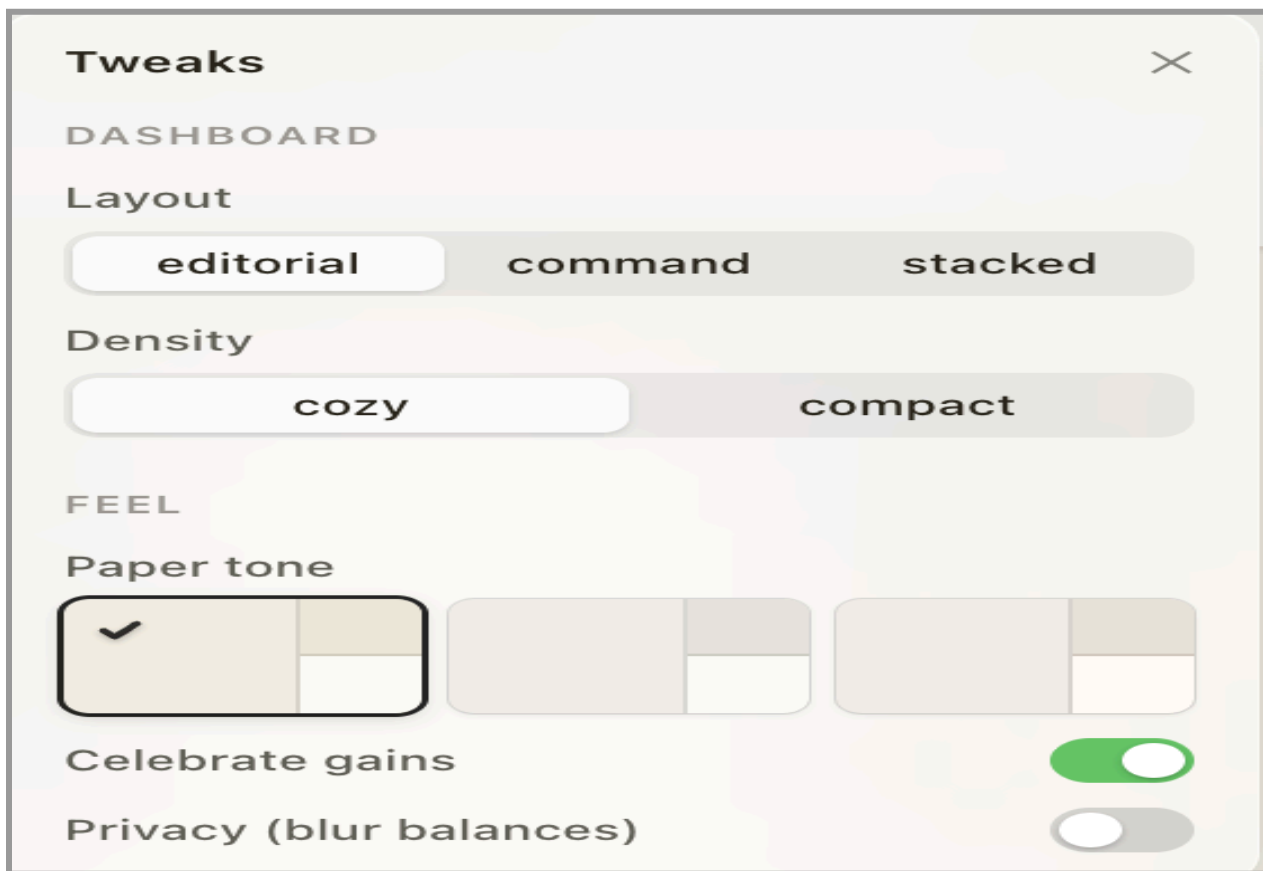
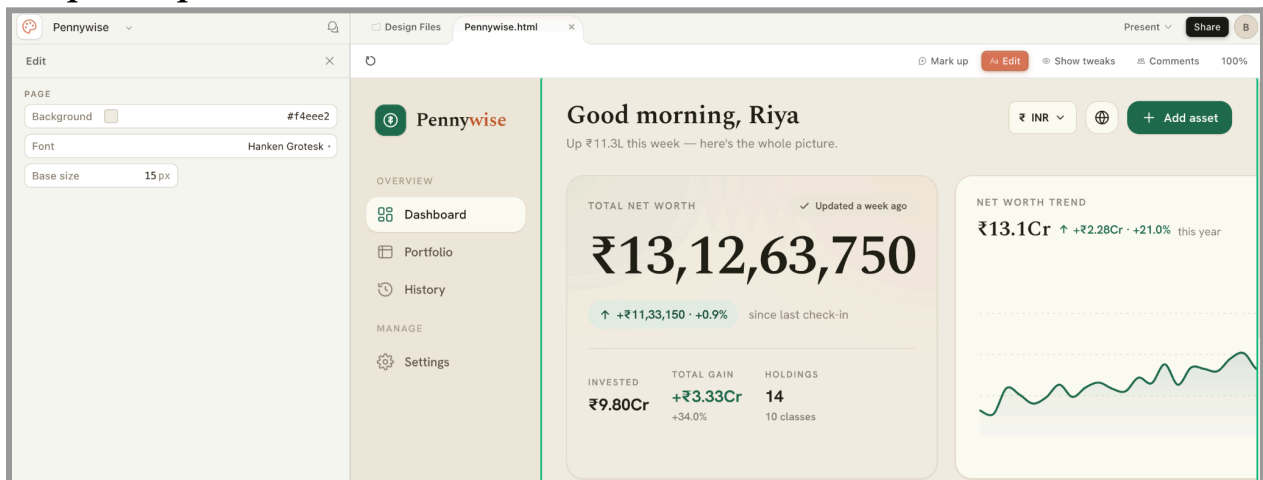


## 1.6 Review and tweak your design

When it finishes, you'll have a clickable prototype — a real dashboard where adding an asset updates the net worth, currency switches between USD and INR, and the pages all work.

- Click around: add a sample asset, switch currency, open the holdings and allocation views. Confirm it behaves like a real app.
- To change anything, just tell Claude in plain language (e.g. “*add a subtle animation to the dashboard*”) or use the **Tweak** and **Edit** buttons on the right. There is no need to be technical — you only need to describe what you want.

## Sample Output:

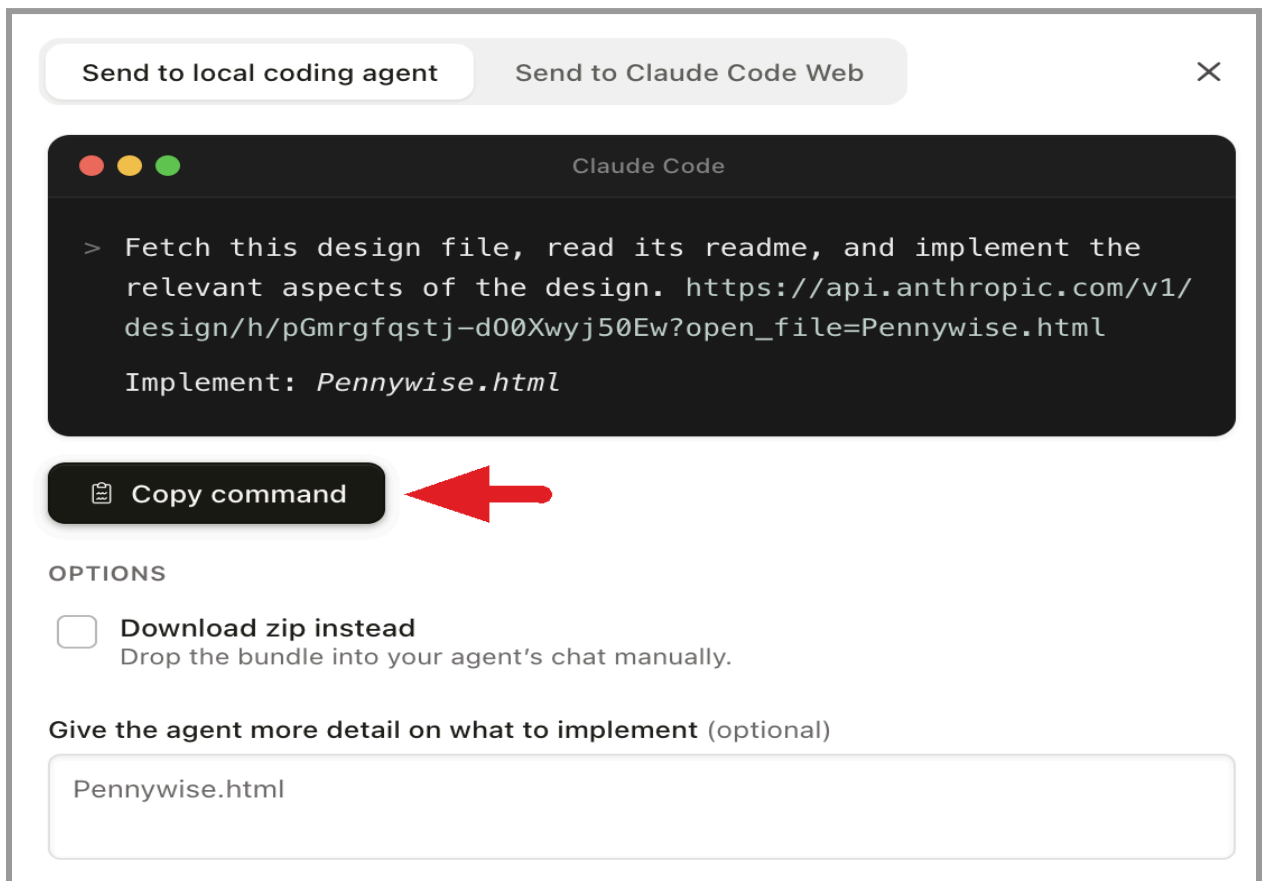
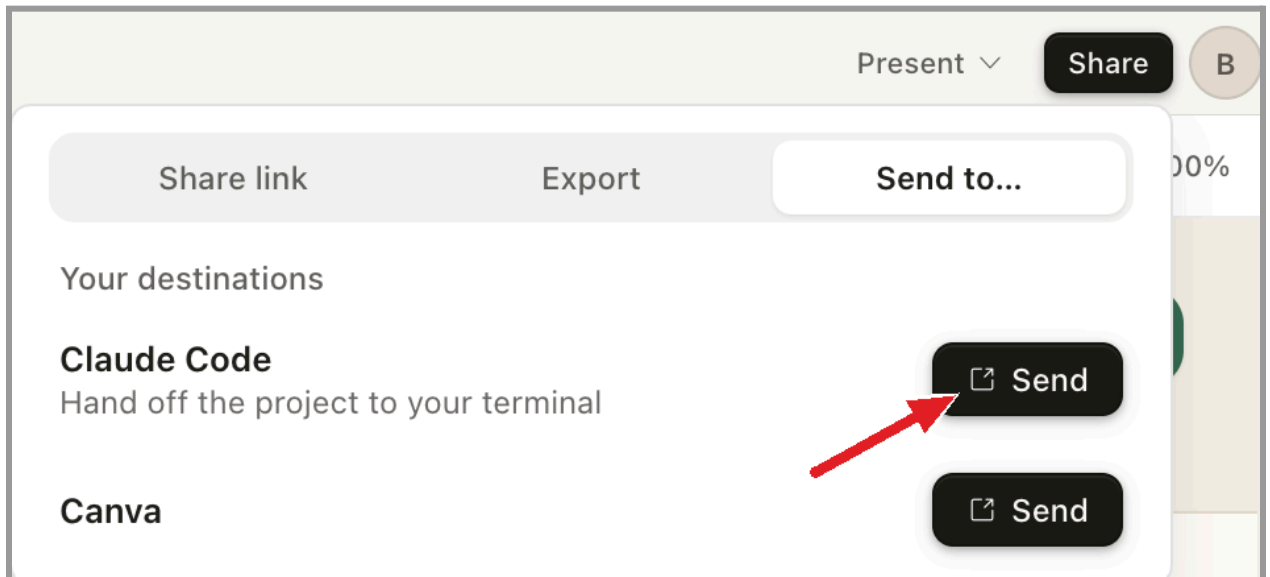


## Part 2: Send the Design to Claude Code & Build the App

The design is only a design — it looks and feels real, but it can't be deployed yet. Now we hand it to Claude Code, which turns it into an actual web app on your computer.

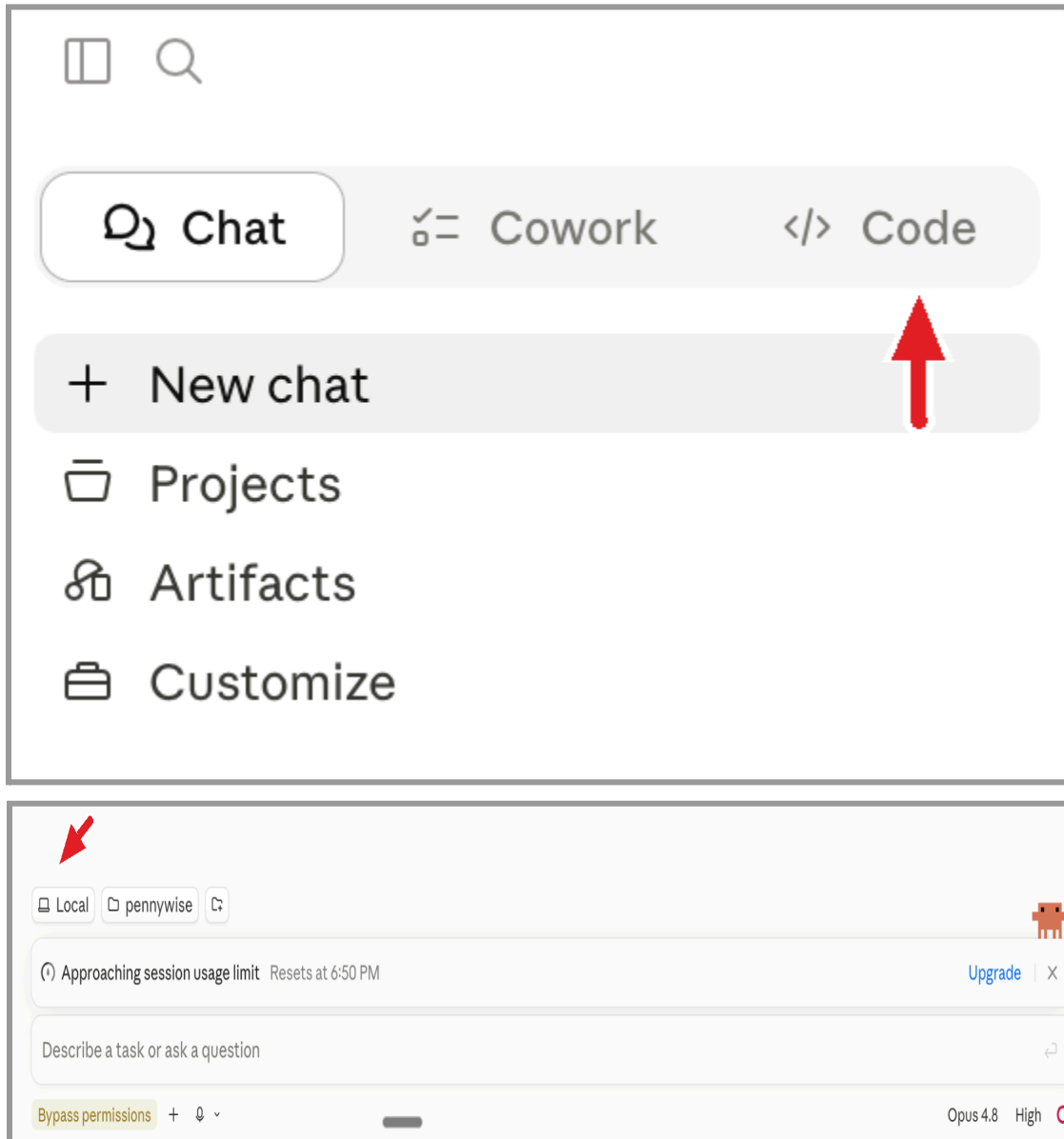
### 2.1 Send the design to Claude Code

- In Claude Design, click **Share**, then **Send to** → **Send to Claude Code**.
- You'll be offered Claude Code on the web or a command to use locally. Click **Copy Command**. We want to build this on your own computer, so keep that command on your clipboard.



## 2.2 Open the Claude desktop app and switch to Code

- Open the [Claude desktop app](#) on your computer.
- Switch to **Code** mode, and make sure you are set to **Local** (this means Claude works on your own PC — your desktop, your hard disk).



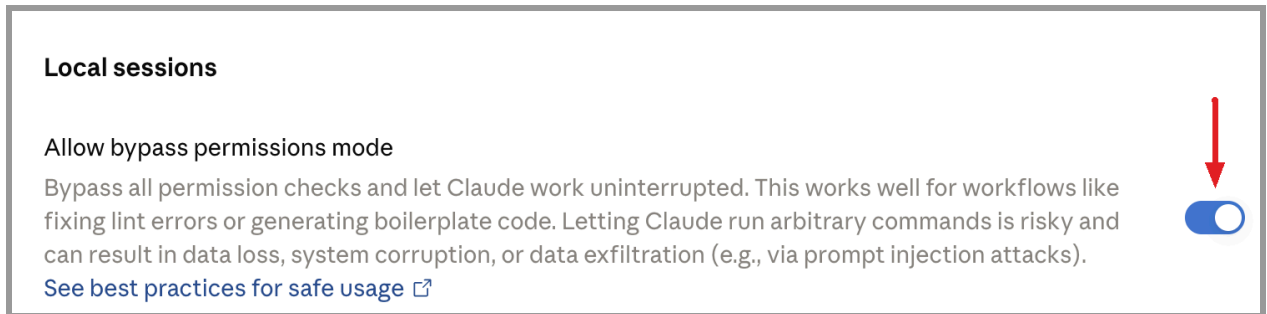
## 2.3 Create a new local Pennywise folder

- Start a **new session** and choose to open a folder.
- Click **New** to create a brand-new, empty folder. Name it **Pennywise** and select it.
- When prompted, click **Trust Workspace**. The folder is empty for now — everything will be built inside it. Claude Code can only access this one folder.



## 2.4 Turn on Bypass Permissions

- In Claude Code **Settings**, enable **Bypass Permissions**.



This simply means Claude won't stop to ask "allow?" for every action while it builds. It's safe here because Claude only has access to your empty Pennywise folder — nothing else on your computer. (It just saves you from clicking "Allow" dozens of times.)

## 2.5 Paste the command and start the build

In the Claude Code chat box, paste the command you copied from Claude Design, then add the build instruction below in the same message.

**Prompt:** *[ Paste the command you copied from Claude Design here ]*

*I want to build a complete web app using this. Pull in everything from the design — all the pages, the layout, and the interactions — and build the first working version of the app.*

- Choose an **Opus** model for the best build quality and speed (you can drop to a Sonnet model later if you want to save credits). Press Enter.

## 2.6 Let Claude build and self-test

Claude Code now fetches all the files and details from your Claude Design project, picks a sensible tech stack on its own, and starts building. It will write the code, then open a browser and test the app itself — all automatically.

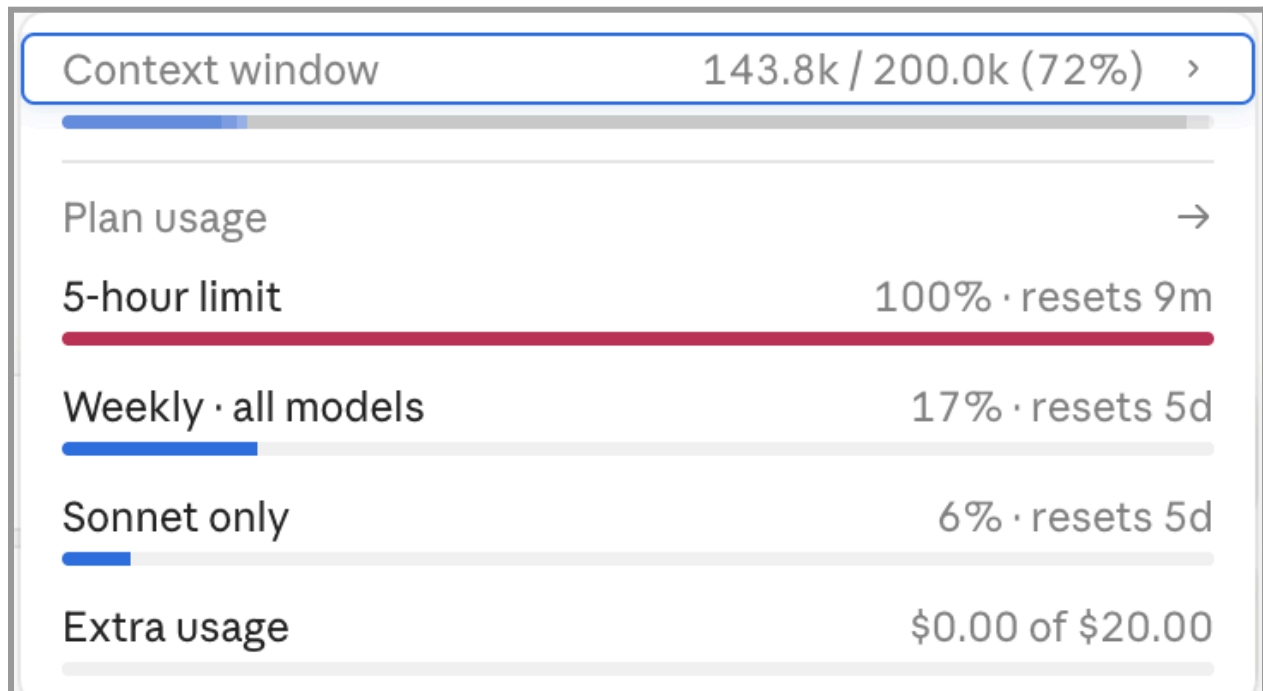
### **This takes time — and that's normal**

Expect 7–10+ minutes for the first build. The first iteration cost Akshat about \$3. You don't need to sit and watch — it keeps running while you do other things.

Watch the small donut icon for your context window and the running token/cost counter. When the build finishes, Claude will show you a working app at a local address such as `localhost:3456` — open it in your browser to see your app running on your own machine.

## 2.7 Reset the context window before the next big step

As Claude works, the context window fills up (watch the donut icon). The fuller it gets, the lower the quality of responses — so before the next large step, save the project state and start fresh. This is a key professional habit.

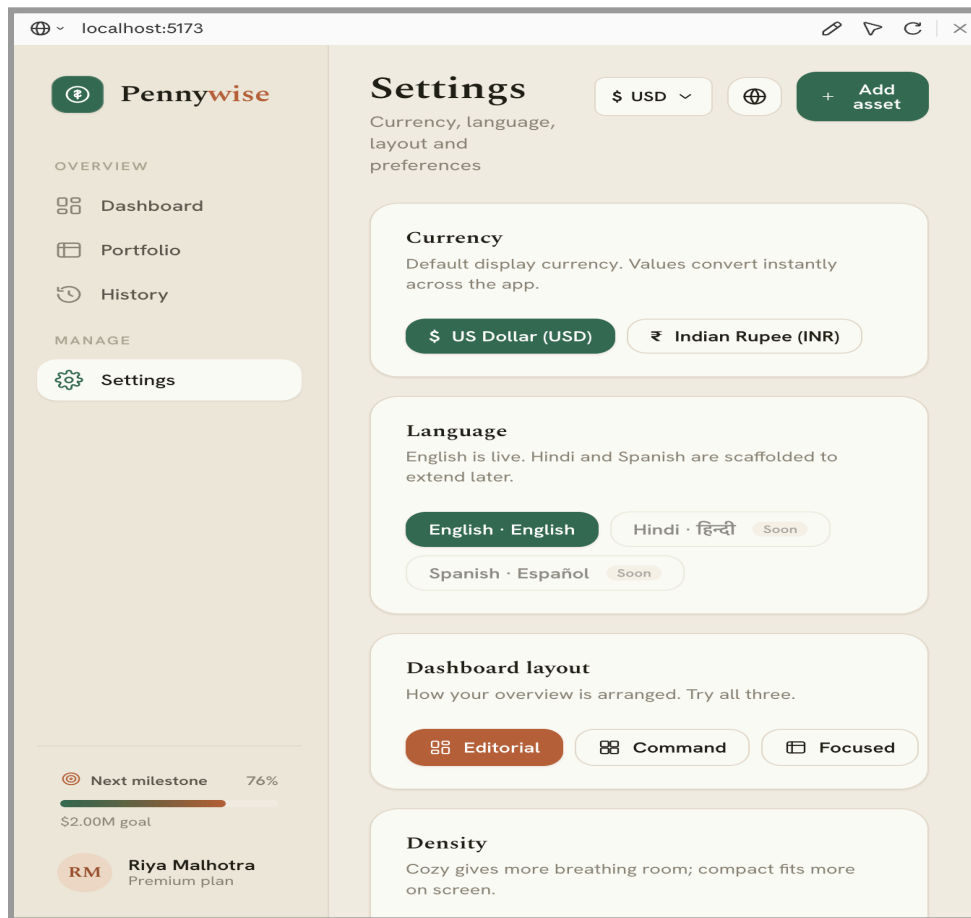


In the current session, ask Claude to write a project memory file:

**Prompt:** *Create a **CLAUDE.md** file and store everything we have done so far — the project overview, the tech stack you chose, the file structure, and what has been built — so that a new session has full context.*

- Once the **CLAUDE.md** file is created, click **New Session**. Make sure you are still in the same **Pennywise** folder — the new session automatically reads **CLAUDE.md** and all your

files, so it keeps full memory of the project with a clean, fast context window.



## Part 3: Make Pennywise Browser-First & Add Onboarding

Right now the app is full of dummy data and isn't ready for real users. We'll make two changes that also make it trivially easy to deploy: (1) make it browser-first, so every user's data lives only in their own browser — no server, no database, nothing shared with you; and (2) add an onboarding flow so a first-time user enters their own data instead of seeing fake numbers.

### 3.1 Start in your fresh session

- You should already be in a **new session** inside the **Pennywise** folder (from step 2.7). If not, start one now — same folder.

### 3.2 Give the browser-first + onboarding prompt

Paste the prompt below. It asks Claude to plan first, then build.

**Prompt:** Build the Pennywise app as a browser-first application. I want to be able to host it on Netlify, so a user can just go to a URL, and all of their data should live entirely in their own browser — nothing stored on a server.

*Also add an onboarding flow. Right now the app is full of dummy data, which doesn't make sense for someone opening it for the first time. A new user should be welcomed with a simple onboarding process where they enter their own basic information, and from there they can see and update their data.*

*Create the simplest possible implementation plan for this, then start executing it.*

### 3.3 Let Claude plan and execute

- Claude will create a to-do list, then start working through it automatically — building the onboarding screens and switching the app to store data in the browser.
- It will take roughly 10–20 minutes and use more credits. Let it run; it will test the app in the browser as it goes. (Tip: if you ever want it to use a specific file as a reference, press @ in the chat to point it at that file.)

### 3.4 Finalise and get it ready to deploy

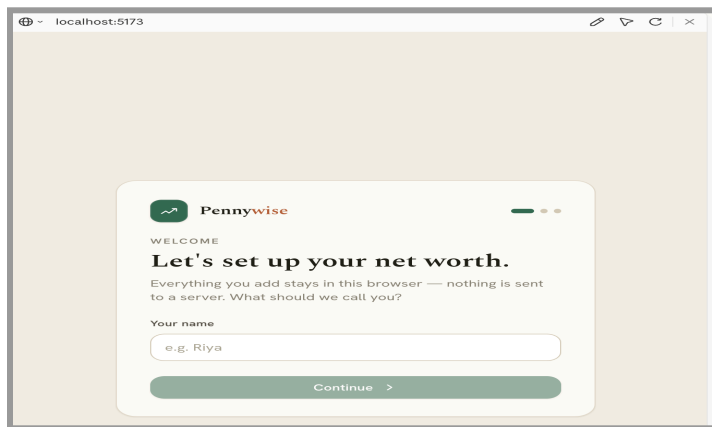
When the build settles, run one more instruction to make sure everything is complete and to find out exactly what to upload to Netlify.

**Prompt:** *Make sure the browser-first build is complete and get it ready for deploying. Then run a final test of the full flow — onboarding, adding assets, and the dashboard — and tell me exactly which folder I should upload to Netlify.*

- **Note the folder Claude tells you to deploy.** For a browser-first app this is usually a build output folder (often called *dist* or *build*) inside your Pennywise folder. You'll drag this exact folder into Netlify in Part 4.

### 3.5 Test the onboarding flow locally

- Open the local address Claude gives you. You should now see a Get Started / onboarding screen instead of dummy data.
- Walk through it: enter your name, pick a currency (USD or INR), and add a couple of holdings. Confirm the dashboard, trends, and allocation all update with your numbers.

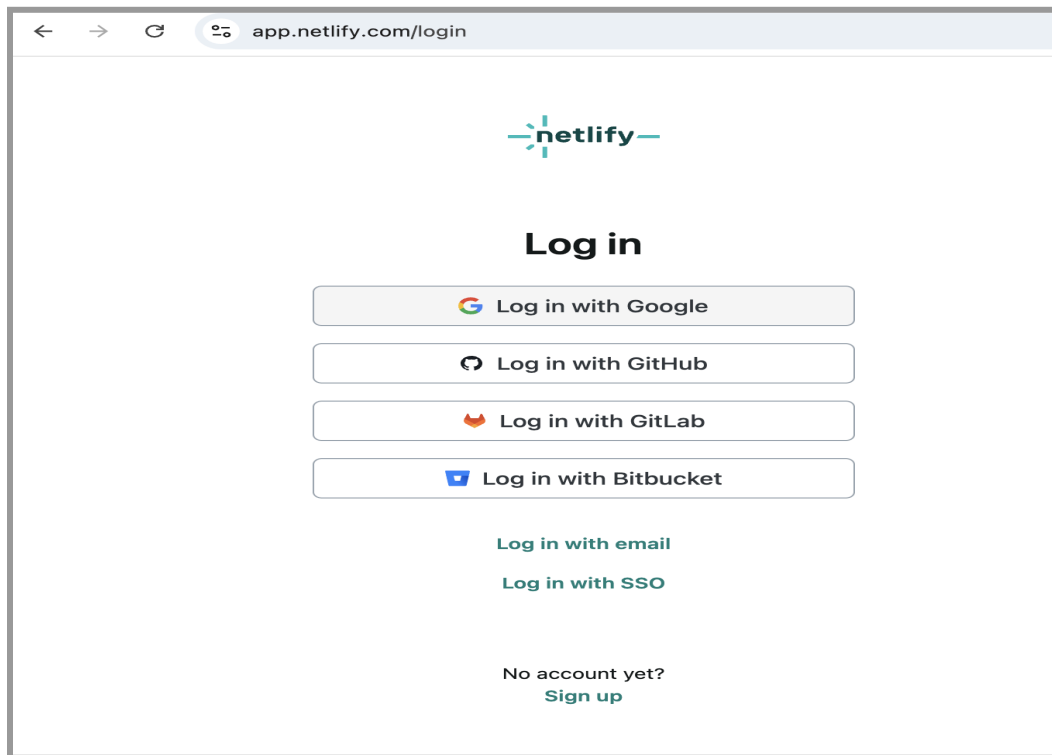


## Part 4: Deploy Pennywise Live on Netlify

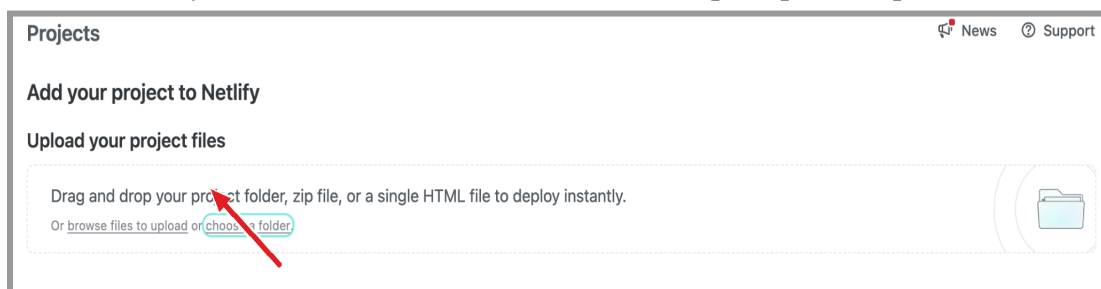
This is the moment your app goes from “running on my computer” to “live on the internet with a real URL anyone can open.” Akshat’s promise: deployment is not hard. On Netlify it is genuinely a drag-and-drop.

### 4.1 Open Netlify and add a new project

- Go to [netlify.com](https://netlify.com) and sign in.



- Click **Add new project**. You’ll see two ways to deploy: connect a Git repository, or deploy a folder directly. We’ll use the folder method — the simplest possible path.



### 4.2 Drag and drop your folder

- Choose the **deploy manually / drag-and-drop** option, then click **Browse files** (or just drag from your file explorer).
- Select the folder Claude told you to deploy in step 3.4 (the build/*dist* folder inside Pennywise) and drop it onto Netlify.

- Netlify starts deploying automatically — there is nothing else to configure.

### 4.3 Open your live URL

- Within a minute or two, Netlify shows “Initializing” → then a live production deploy with an auto-generated domain name (something like pennywise-xyz.netlify.app).
- **Click the URL.** Your app is now live on the internet — open it, and share the link with a friend to test it on their browser too.

**Sample Output:** <https://pennywise11-1c3f48.netlify.app/>

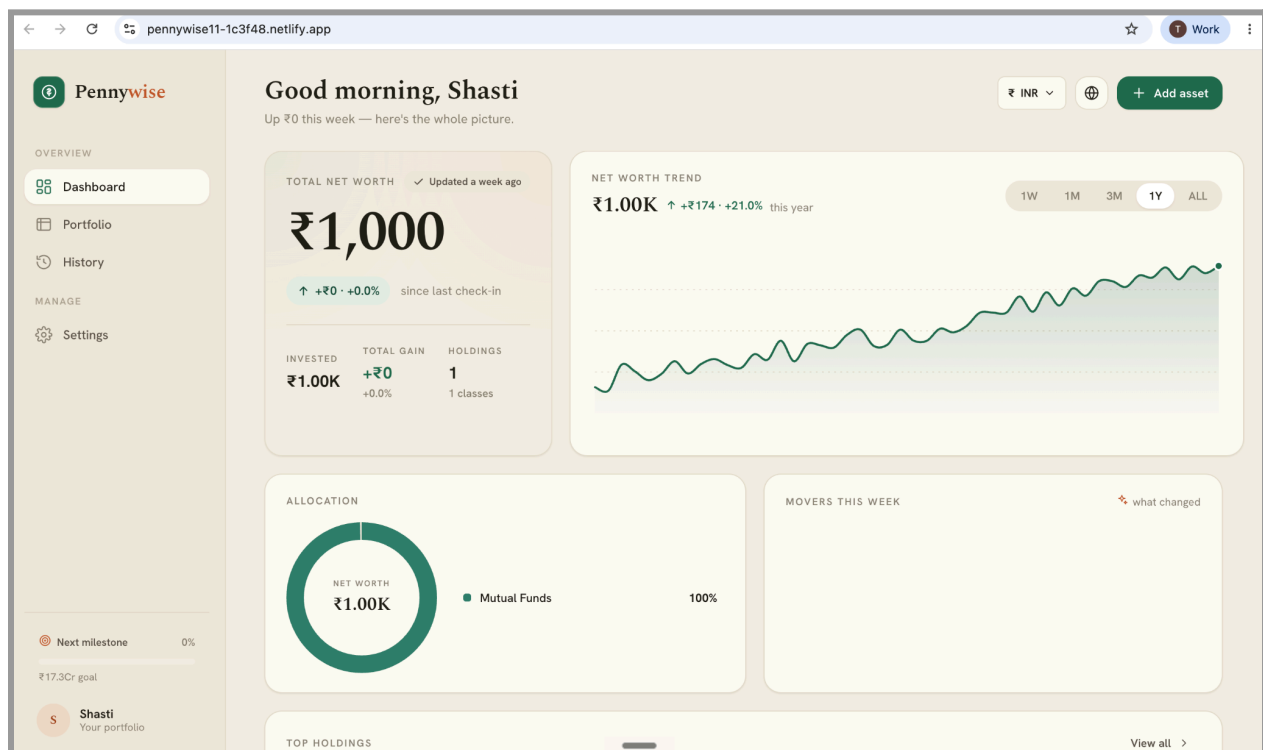
### 4.4 Confirm the data lives in the browser

- On the live site, complete onboarding and add some holdings.
- Close the tab completely, then reopen the URL. Your data is still there — because it’s saved in your browser, not on any server. Open the same link on a different device and you’ll get a fresh start, which proves each person’s data stays private to their own browser.

### 4.5 (Optional) Rename your domain

- To make the URL nicer, go to **Domain management** in Netlify and change the site name (e.g. *pennywise-app.netlify.app*). You can also buy and connect a custom domain here later if you want.

**Sample Output:** <https://pennywise11-1c3f48.netlify.app/>



## You're Done!

---

Here is what you just accomplished:

- You designed a polished, interactive product (Pennywise) from a single detailed prompt in Claude Design.
- You sent that design to Claude Code and had it build a real, working web app — without writing any code.
- You refactored it into a browser-first app with an onboarding flow, so it's private by default and ready for real users.
- You deployed it live on Netlify and got a shareable URL — a real product, on the internet, built by you.

You just ran the full professional loop: idea → design → build → ship. The exact same four steps will let you build and launch almost any web app you can describe. Remember Akshat's framing: you don't need to code — you need to think clearly and describe what you want, step by step. And bugs are part of the process; when something breaks, you just tell Claude what's wrong and ask it to fix it.

## Where to Go Next (Beyond This Workbook)

---

In the session, Akshat went further than we do here. Once you're comfortable with the core flow above, these are the natural next steps to explore on your own (all shown in the Day 8 recording):

- **Continuous deployment with GitHub.** Ask Claude Code to create a [GitHub](#) repository for Pennywise, then connect that repo to Netlify. After that, every change you ask Claude to “send to GitHub” redeploys your live site automatically — no more manual drag-and-drop.
- **Add a database.** Grab a backend skill (for example a Supabase skill from [skills.sh](#)), tell Claude to install it, and ask it to add a database — useful when you want data to sync across devices instead of living only in the browser.
- **Build a “feature-builder” skill.** Ask Claude to create a reusable skill for Pennywise so that adding a new feature (like a SIP calculator) becomes a single command — the skill already knows your whole project, so you don't have to re-explain it each time.

## Quick Tips

---

- **Let Claude cook.** Designs and builds take minutes, not seconds. Start it, walk away, and come back — it keeps running on your machine.
- **Watch your spend.** Keep an eye on [Settings → Usage](#) and the in-build cost counter. Top up credits before you start so you don't get locked out mid-build.
- **Reset context between big steps.** When the context donut fills up, save a CLAUDE.md and start a new session in the same folder for sharper responses.

- **Detail beats brevity.** The long, specific design prompt is what makes the output great. When in doubt, describe more, not less.
- **Errors are expected — and easy.** You won't fix bugs by hand. Paste the error or describe what's wrong, and ask Claude to fix it. If a deploy looks off, give it a moment — it sometimes finishes a few seconds after it says it has.